Day – 4 Weekly Practical and Troubleshooting.

Task – 1 Simulate a WAN setup connecting.

- Step 1: Network Topology Overview
- To simulate two branch offices connecting securely via a VPN tunnel while accessing the internet through NAT, your Packet Tracer topology should look like this:
- Branch A: Contains an internal local PC connected to a switch, which in turn connects to Router A (the edge router).
- Branch B: Contains an internal local PC connected to a switch, which connects to Router B (the edge router).
- WAN / Internet Cloud: Connect GigabitEthernet 0/1 on Router A and Router B to a central cloud or switch representing the public internet/service provider.
- External Server: Connect a public web server to the internet cloud to test outbound NAT internet access.
- Step 2: IP Addressing Scheme
- Branch A LAN: 192.168.1.0/24
- PC A: 192.168.1.10 (Gateway: 192.168.1.1)
- Router A Inside Interface (GigabitEthernet 0/0): 192.168.1.1
- Branch B LAN: 192.168.2.0/24
- PC B: 192.168.2.10 (Gateway: 192.168.2.1)
- Router B Inside Interface (GigabitEthernet 0/0): 192.168.2.1
- Public WAN / Interconnect: 203.0.113.0/24
- Router A Outside Interface (GigabitEthernet 0/1): 203.0.113.1
- Router B Outside Interface (GigabitEthernet 0/1): 203.0.113.2
- GRE Tunnel Network: 192.168.100.0/30
- Router A Tunnel 0: 192.168.100.1
- Router B Tunnel 0: 192.168.100.2
- Step 3: Configuring Router A (Branch A Edge Router)
- Open Router A, go to the CLI tab, and enter the following commands to configure interfaces, the GRE tunnel, static routes for the VPN, and PAT for internet access:
- Router> enable
- Router# configure terminal
- Router(config)# hostname RouterA
- ! Configure Internal LAN Interface
- RouterA(config)# interface gigabitEthernet 0/0
- RouterA(config-if)# ip address 192.168.1.1 255.255.255.0
- RouterA(config-if)# ip nat inside

- RouterA(config-if)# no shutdown
- RouterA(config-if)# exit
- ! Configure External WAN Interface
- RouterA(config)# interface gigabitEthernet 0/1
- RouterA(config-if)# ip address 203.0.113.1 255.255.255.0
- RouterA(config-if)# ip nat outside
- RouterA(config-if)# no shutdown
- RouterA(config-if)# exit
- ! Configure GRE Tunnel Interface
- RouterA(config)# interface tunnel 0
- RouterA(config-if)# ip address 192.168.100.1 255.255.255.252
- RouterA(config-if)# tunnel source gigabitEthernet 0/1
- RouterA(config-if)# tunnel destination 203.0.113.2
- RouterA(config-if)# exit
- ! Route for Remote Branch via Tunnel
- RouterA(config)# ip route 192.168.2.0 255.255.255.0 tunnel 0
- ! Default Route for Internet Traffic via WAN Gateway
- RouterA(config)# ip route 0.0.0.0 0.0.0.0 203.0.113.254
- ! Configure PAT (NAT Overload)
- RouterA(config)# access-list 1 permit 192.168.1.0 0.0.0.255
- RouterA(config)# ip nat inside source list 1 interface gigabitEthernet 0/1 overload
- RouterA(config)# exit
- RouterA# write memory
- Step 4: Configuring Router B (Branch B Edge Router)
- Open Router B, go to the CLI tab, and execute the matching configuration for the second branch:
- Router> enable
- Router# configure terminal
- Router(config)# hostname RouterB
- ! Configure Internal LAN Interface
- RouterB(config)# interface gigabitEthernet 0/0
- RouterB(config-if)# ip address 192.168.2.1 255.255.255.0
- RouterB(config-if)# ip nat inside
- RouterB(config-if)# no shutdown
- RouterB(config-if)# exit
- ! Configure External WAN Interface
- RouterB(config)# interface gigabitEthernet 0/1
- RouterB(config-if)# ip address 203.0.113.2 255.255.255.0
- RouterB(config-if)# ip nat outside
- RouterB(config-if)# no shutdown
- RouterB(config-if)# exit

- ! Configure GRE Tunnel Interface
- RouterB(config)# interface tunnel 0
- RouterB(config-if)# ip address 192.168.100.2 255.255.255.252
- RouterB(config-if)# tunnel source gigabitEthernet 0/1
- RouterB(config-if)# tunnel destination 203.0.113.1
- RouterB(config-if)# exit
- ! Route for Remote Branch via Tunnel
- RouterB(config)# ip route 192.168.1.0 255.255.255.0 tunnel 0
- ! Default Route for Internet Traffic via WAN Gateway
- RouterB(config)# ip route 0.0.0.0 0.0.0.0 203.0.113.254
- ! Configure PAT (NAT Overload)
- RouterB(config)# access-list 1 permit 192.168.2.0 0.0.0.255
- RouterB(config)# ip nat inside source list 1 interface gigabitEthernet 0/1 overload
- RouterB(config)# exit
- RouterB# write memory

Task – 2 Config the PAT on the router.

- Step 1: Configuring the Access Control List (ACL)
- To define which internal devices are permitted to share the public IP address, create a standard access list on the router:
- RouterA(config)# access-list 1 permit 192.168.1.0 0.0.0.255
- Step 2: Enabling PAT with the Overload Keyword
- Bind the access list to the router's external public-facing interface (GigabitEthernet 0/1) and apply the overload keyword so multiple internal devices can translate simultaneously using that single public IP:
- RouterA(config)# ip nat inside source list 1 interface gigabitEthernet 0/1 overload
- RouterA(config)# exit
- RouterA# write memory
- Step 3: Verifying PAT with Three Internal Devices
- Generate Traffic from the Internal PCs: Open the command prompt on three separate internal devices (e.g., PC 0 at 192.168.1.10, PC 1 at 192.168.1.20, and PC 2 at 192.168.1.30) and ping an external public destination (such as the web server at 203.0.113.100):
- C:\> ping 203.0.113.100
- Inspect the Router's NAT Table: Access the router CLI and run the verification command to check the translation table:
- RouterA# show ip nat translations

- Confirming Address Mapping: You will see entries showing each unique internal device IP and port mapped to the single public interface IP (203.0.113.1), confirming that PAT overload is successfully allowing all three internal devices to browse the external network concurrently.

Task – 3 Misconfig the VPN Tunnel's between two routers.

- Step 1: Intentionally Misconfiguring the Pre-Shared Key
- To simulate a common configuration error where VPN tunnels fail due to mismatched authentication parameters, change the pre-shared key on Router A so it no longer matches Router B.
- Open Router A CLI and enter the following commands:
- RouterA> enable
- RouterA# configure terminal
- RouterA(config)# crypto isakmp policy 10
- RouterA(config-isakmp)# authentication pre-share
- RouterA(config-isakmp)# exit
- RouterA(config)# crypto isakmp key WRONG_KEY address 203.0.113.2
- RouterA(config)# exit
- RouterA# write memory
- Step 2: Identifying the Connectivity Issue
- When the pre-shared keys mismatch, Phase 1 ISAKMP (Internet Security Association and Key Management Protocol) negotiation fails, meaning the secure tunnel cannot establish.
- Test Connectivity: Try pinging from Branch A's PC to Branch B's PC. The pings will fail because the traffic cannot be encrypted and sent through the tunnel.
- Check ISAKMP Security Associations: Open Router A or Router B CLI and run the command to inspect the crypto ISAKMP state:
- RouterA# show crypto isakmp sa
- What you will see: The state will show as MM_NO_STATE, Init, or QM_IDLE failing to reach Active (or no active SA will be listed at all), indicating that authentication failed during the handshake due to the incorrect key.
- Step 3: Inspecting the Configuration
- To confirm the mismatched keys, review the running configuration on both routers using the show run command:
- RouterA# show run | include crypto
- Output on Router A:
- crypto isakmp key WRONG_KEY address 203.0.113.2

- Comparing this with Router B's configuration (show run | include crypto), you will notice the key mismatch (WRONG_KEY vs CORRECT_SECRET_KEY).
- Step 4: Fixing the Connectivity Issue
- To resolve the issue, update the pre-shared key on Router A so it matches the key configured on Router B:
- RouterA> enable
- RouterA# configure terminal
- RouterA(config)# no crypto isakmp key WRONG_KEY address 203.0.113.2
- RouterA(config)# crypto isakmp key CORRECT_SECRET_KEY address 203.0.113.2
- RouterA(config)# exit
- RouterA# write memory
- Step 5: Verifying the Fix
- Clear any existing failed crypto sessions to force a fresh negotiation:
- RouterA# clear crypto isakmp sa
- Re-run the verification command on Router A:
- RouterA# show crypto isakmp sa
- Expected Output: The state will now display as QM_IDLE or ACTIVE, confirming that Phase 1 negotiation has successfully authenticated and established the secure tunnel.
- Test connectivity by pinging between the branch PCs once more to verify that traffic is successfully flowing across the tunnel.

Task – 4 Based on scenario.

- Step 1: Checking the Routing Table
- When internet access works (thanks to PAT) but branch-to-branch traffic fails, the first thing to check is whether the router actually knows *how* to reach the remote branch network.
- Run this command on Router A:
- RouterA# show ip route
- What to Look For:
- Look for a route pointing to the remote branch subnet (e.g., 192.168.2.0/24).
- The Issue: If you see a default route (0.0.0.0/0) going to the internet gateway but *no* specific static route pointing 192.168.2.0/24 to tunnel 0, the router is dumping branch-bound traffic out to the public internet instead of encapsulating it through the VPN tunnel.
- Step 2: Checking for NAT Overlap (The Hidden Culprit)

- If the route exists, the next most common culprit is a misconfigured NAT Access Control List (ACL). If your PAT statement matches too much traffic, it can intercept traffic destined for the VPN tunnel.
- Run these commands on Router A:
- RouterA# show access-lists
- RouterA# show ip nat translations
- What to Look For:
- The Issue: If your NAT ACL is written too broadly (for example, using a wildcard mask that matches all private traffic or the entire local network space), the router treats traffic destined for the remote branch (192.168.2.0/24) as internet traffic and applies PAT overload to it. PAT translates the private IP to the public WAN IP and shoves it out the physical WAN port (GigabitEthernet 0/1) instead of pushing it into the GRE tunnel. The receiving end drops it because it arrives unencrypted and untunneled.
- Step 3: Fixing the Routing and NAT Configuration
- To resolve this issue, you need to ensure two things:
- The routing table explicitly steers remote branch traffic into the tunnel.
- The NAT configuration explicitly *excludes* traffic bound for the remote branch.
- 1. Add or Correct the Missing Static Route
- If the route was missing on Router A, add it:
- RouterA> enable
- RouterA# configure terminal
- RouterA(config)# ip route 192.168.2.0 255.255.255.0 tunnel 0
- 2. Fix the NAT Overlap Using an Extended ACL (or Route-Map)
- To prevent PAT from capturing traffic meant for the VPN, update your ACL so it only translates traffic destined for the public internet, or use an extended ACL to exclude the remote subnet.
- For example, using a standard ACL combined with a route-map or refining the permit statement:
- ! Remove the overly broad ACL
- RouterA(config)# no access-list 1
- ! Create a refined ACL that permits local LAN traffic but restricts NAT
- RouterA(config)# access-list 1 permit 192.168.1.0 0.0.0.255
- ! Ensure traffic destined for the other branch bypasses NAT by adjusting your routing/NAT flow,
- ! or use an extended ACL for NAT matching:
- RouterA(config)# ip access-list extended NAT-TRAFFIC
- RouterA(config-ext-nacl)# deny ip 192.168.1.0 0.0.0.255 192.168.2.0 0.0.0.255
- RouterA(config-ext-nacl)# permit ip 192.168.1.0 0.0.0.255 any
- RouterA(config-ext-nacl)# exit
-

- ! Re-apply NAT using the extended ACL
- RouterA(config)# no ip nat inside source list 1 interface gigabitEthernet 0/1 overload
- RouterA(config)# ip nat inside source list NAT-TRAFFIC interface gigabitEthernet 0/1 overload
- RouterA(config)# exit
- RouterA# write memory
- Step 4: Verifying the Fix
- Clear existing translation tables to flush stale states:
- RouterA# clear ip nat translation *
- Test connectivity from Branch A's PC to Branch B's PC:
- C:\> ping 192.168.2.10
- Successful echo replies confirm that branch traffic is now correctly entering the GRE tunnel rather than being erroneously caught by PAT.

Task – 5 Take WAN, VPN and NAT scenario.

- 1. Reviewing the Swapped Configuration (show run)
- After swapping configuration files with a classmate who built the identical dual-branch WAN, VPN, and NAT topology, I uploaded their config to my router and noticed that while internet browsing (PAT) worked perfectly, branch-to-branch pings over the GRE tunnel failed completely.
- Instead of guessing the cause, I started by running a complete configuration review using:
- RouterA# show running-config
- What I Found:
- Scanning through the interface and tunnel sections of their show run output, I spotted this block on their Tunnel 0 interface:
- interface Tunnel0
- ip address 192.168.100.1 255.255.255.252
- tunnel source GigabitEthernet0/1
- tunnel destination 203.0.113.2
- Everything looked correct at first glance, but when I checked the physical WAN interface IP addresses using show ip interface brief, I discovered that their physical outside interface (GigabitEthernet 0/1) was actually assigned 203.0.113.5, meaning the tunnel source definition was pointing to an incorrect IP/interface, or the peer's destination IP didn't match what the other router was listening on.
- 2. Using Diagnostic Commands (show and debug)

- To confirm why the tunnel packets were dropping, I used live diagnostic commands:
- Check Tunnel Interface Status:
- RouterA# show interface tunnel 0
- Finding: The output showed Tunnel0 is up, line protocol is DOWN. A down line protocol on a GRE tunnel explicitly indicates that the router cannot establish a valid path or encapsulation handshake with the far-end peer (due to mismatched or unreachable source/destination parameters).
- Inspect Crypto / Tunnel Packets: To verify if traffic was even reaching the tunnel interface or getting dropped, I turned on real-time packet debugging:
- RouterA# debug tunnel
- Finding: The debug console immediately flooded with messages like Tunnel0:- destination 203.0.113.2 unreachable or encapsulation drop counters incrementing rapidly under show interfaces tunnel 0. This proved the router was trying to send traffic out an invalid destination path.
- 3. Fixing the Misconfiguration
- To resolve the issue, I corrected the tunnel destination command on both routers so they accurately pointed to each other's live public WAN interface addresses:
- RouterA> enable
- RouterA# configure terminal
- RouterA(config)# interface tunnel 0
- RouterA(config-if)# no tunnel destination 203.0.113.2
- RouterA(config-if)# tunnel destination 203.0.113.5 ! (Updating to my classmate's correct peer WAN IP)
- RouterA(config-if)# exit
- RouterA# write memory
- 4. Verifying the Resolution
- After applying the fix, I turned off debugging (no debug all) and re-verified the tunnel status:
- Checked the tunnel line protocol:
- RouterA# show interface tunnel 0
- Result: The status now cleanly read Tunnel0 is up, line protocol is UP.
- Tested branch-to-branch connectivity via ping:
- C:\> ping 192.168.2.10
- Result: Successful echo replies confirmed that the GRE tunnel was fully operational and passing traffic between the two branch offices.